

API de Banca en Línea

Esta API simula el backend de una aplicación bancaria. Tu app de Android (con Firebase Auth ya implementado) la consume desde Retrofit para crear cuentas, registrar beneficiarios y mover dinero entre usuarios.

Este documento asume que ya sabes configurar Retrofit, crear data classes y montar un repositorio o ViewModel. Aquí solo encontrarás:

- El contrato HTTP de cada endpoint.
- Cómo autenticar las peticiones con un interceptor.
- Cómo manejar los errores que devuelve la API.

Tabla de contenido

1. [Antes de empezar](#)
2. [Autenticación: AuthInterceptor](#)
3. [Manejo de errores](#)
4. [Convención del dinero \(centavos\)](#)
5. [Formato de fechas](#)
6. [Endpoints de cuenta](#)
7. [Endpoints de beneficiarios](#)
8. [Endpoints de transacciones](#)
9. [Tabla de códigos de error](#)
10. [Ejemplo de flujo completo](#)

Antes de empezar

URL base

```
https://us-central1-TU-PROYECTO-ID.cloudfunctions.net/api/
```

Reemplaza `TU-PROYECTO-ID` por el ID de tu proyecto de Firebase. **La barra final es obligatoria** para Retrofit (si la quitas, las llamadas a paths como `"account"` fallarán con errores raros).

Una cuenta por usuario

La API impone una regla importante: **cada usuario solo puede tener una cuenta**. Si intentas crear una segunda, recibirás un `409`.

Toda colección "bancaria" pasa por la API

No intentes leer ni escribir `accounts`, `transactions` o `beneficiaries` directamente con el SDK de Firestore desde tu app: las reglas de seguridad lo bloquean. La única forma de tocar esos datos es a través de esta API.

Autenticación: AuthInterceptor

Cada petición a la API debe llevar un **ID token de Firebase Auth** en el header:

```
Authorization: Bearer <idToken>
```

En lugar de adjuntar el token manualmente en cada llamada, lo correcto en Android es delegarlo a un **interceptor de OkHttp**: una pieza que corre en cada request, le pide a Firebase el token actual del usuario y lo agrega al header antes de que la petición salga del dispositivo.

Código del interceptor

```

package com.tuapp.banca.network

import com.google.android.gms.tasks.Tasks
import com.google.firebase.auth.FirebaseAuth
import okhttp3.Interceptor
import okhttp3.Response
import java.io.IOException

class AuthInterceptor : Interceptor {
    override fun intercept(chain: Interceptor.Chain): Response {
        val user = FirebaseAuth.getInstance().currentUser
        ?: throw IOException("No hay usuario autenticado.")

        // Tasks.await() bloquea el hilo del interceptor (que vive en su
        // propio thread pool de OkHttp), no el hilo principal.
        //
        // El parámetro `false` significa: usa el token cacheado si todavía
        // es válido; si expiró, Firebase lo renueva automáticamente.
        val token = try {
            Tasks.await(user.getIdToken(false)).token
        } catch (e: Exception) {
            throw IOException("No se pudo obtener el ID token de Firebase.", e)
        } ?: throw IOException("ID token vacío.")

        val request = chain.request().newBuilder()
            .addHeader("Authorization", "Bearer $token")
            .build()

        return chain.proceed(request)
    }
}

```

Cómo conectarlo a tu OkHttpClient

```

val client = OkHttpClient.Builder()
    .addInterceptor(AuthInterceptor())
    .build()

val retrofit = Retrofit.Builder()
    .baseUrl(BASE_URL)
    .client(client)
    .addConverterFactory(GsonConverterFactory.create())
    .build()

```

Con esto montado, **nunca más** tocas el token desde tu código. Cada endpoint se llama como cualquier otro método de Retrofit y el interceptor inyecta el `Authorization` por debajo.

Dependencia adicional

Recuerda que `Tasks.await()` necesita esta dependencia (probablemente ya la tienes para Firebase, pero por si acaso):

```
implementation("com.google.android.gms:play-services-tasks:18.2.0")
```

Manejo de errores

Todas las respuestas de error de la API tienen exactamente esta forma:

```

{
  "error": "codigo_corto",
  "message": "Descripción legible del error."
}

```

- `error` — un **código corto** (`insufficient_funds`, `no_account`, etc.) útil para reaccionar de forma específica en tu UI con un `when`.
- `message` — texto en español listo para mostrar al usuario.

El problema con Retrofit

Cuando la API responde 4xx/5xx, Retrofit lanza una `HttpException` y el cuerpo del error queda enterrado en `response.errorBody()`. Para no estar parseando ese cuerpo en cada llamada, usa este helper:

```
package com.tuapp.banca.network

import com.google.gson.Gson
import retrofit2.HttpException

// Data class para parsear el body de error de la API.
data class ApiErrorBody(val error: String, val message: String)

// Excepción tipada que tu código sí entiende.
class ApiException(
    val errorCode: String,
    message: String,
    val statusCode: Int
) : Exception(message)

private val gson = Gson()

/**
 * Envuelve una llamada a la API y devuelve Result<T>.
 * Convierte HttpException en ApiException con el código de la API.
 */
suspend fun <T> apiCall(block: suspend () -> T): Result<T> = try {
    Result.success(block())
} catch (e: HttpException) {
    val raw = e.response()?.errorBody()?.string()
    val apiError = try {
        gson.fromJson(raw, ApiErrorBody::class.java)
    } catch (_: Exception) {
        ApiErrorBody("unknown_error", e.message ?: "Error desconocido")
    }
    Result.failure(ApiException(apiError.error, apiError.message, e.code()))
} catch (e: Exception) {
    Result.failure(e)
}
```

Cómo usarlo

```
apiCall { api.getAccount() }
    .onSuccess { account ->
        // ...
    }
    .onFailure { e ->
        if (e is ApiException) {
            when (e.errorCode) {
                "no_account"      -> mostrarPantallaSinCuenta()
                "unauthorized"     -> volverALogin()
                else               -> mostrarError(e.message)
            }
        } else {
            // Problema de red, parseo, etc.
            mostrarError("Sin conexión")
        }
    }
}
```

Convención del dinero (centavos)

Todos los montos van y vienen como enteros (Long) en centavos. Esto evita problemas de precisión con números flotantes (0.1 + 0.2 != 0.3).

Cantidad real	Lo que mandas / recibes
\$1.00	100
\$50.00	5000
\$1,234.56	123456

Para mostrar balances en tu UI, divide entre 100 y formatea con Locale :

```
import java.text.NumberFormat
import java.util.Locale

fun Long.toCurrencyMXN(): String {
    val fmt = NumberFormat.getCurrencyInstance(Locale("es", "MX"))
    return fmt.format(this / 100.0)
}
```

Formato de fechas

Las fechas (createdAt , date , etc.) llegan como objetos de Firestore Timestamp:

```
{ "_seconds": 1717325400, "_nanoseconds": 0 }
```

En tu data class declara un tipo equivalente:

```
import com.google.gson.annotations.SerializedName
import java.util.Date

data class FirebaseTimestamp(
    @SerializedName("_seconds") val seconds: Long,
    @SerializedName("_nanoseconds") val nanoseconds: Int
) {
    fun toDate(): Date = Date(seconds * 1000)
}
```

Endpoints de cuenta

POST /account

Crea la cuenta del usuario autenticado con balance en 0.

- Auth requerida: sí
- Body: ninguno

Respuesta exitosa (201 Created):

```
{
  "id": "xY7kP2mNqR",
  "ownerId": "uid-del-usuario",
  "accountNumber": "4827193056",
  "balance": 0,
  "createdAt": { "_seconds": 1717325400, "_nanoseconds": 0 }
}
```

Errores posibles:

Status	error	Cuándo
401	unauthorized	Token faltante o inválido
409	account_exists	El usuario ya tiene una cuenta (solo se permite una)

Ejemplo:

```
apiCall { api.createAccount() }
  .onSuccess { account -> Log.d("Banca", "Cuenta: ${account.accountNumber}") }
```

GET /account

Devuelve la cuenta del usuario autenticado.

- Auth requerida: sí

Respuesta exitosa (200 OK):

```
{
  "id": "xY7kP2mNqR",
  "ownerId": "uid-del-usuario",
  "accountNumber": "4827193056",
  "balance": 5000,
  "createdAt": { "_seconds": 1717325400, "_nanoseconds": 0 }
}
```

Errores posibles:

Status	error	Cuándo
401	unauthorized	Token faltante o inválido
404	no_account	El usuario aún no ha creado su cuenta

Ejemplo:

```
apiCall { api.getAccount() }
  .onSuccess { account -> textoSaldo = account.balance.toCurrencyMXN() }
```

PUT /account

Fondea la cuenta con dinero ficticio (suma al balance actual).

- Auth requerida: sí
- Body:

```
{ "amount": 5000 }
```

Campo	Tipo	Obligatorio	Descripción
amount	entero positivo	sí	Cantidad a ingresar, en centavos

Respuesta exitosa (200 OK):

La cuenta con el nuevo balance.

```
{
  "id": "xY7kP2mNqR",
  "ownerId": "uid-del-usuario",
  "accountNumber": "4827193056",
  "balance": 10000
}
```

Errores posibles:

Status	error	Cuándo
400	invalid_amount	amount no es entero positivo
401	unauthorized	Token faltante o inválido
404	no_account	El usuario no tiene cuenta

Ejemplo:

```
apiCall { api.fundAccount(FundRequest(amount = 5000)) }
```

Endpoints de beneficiarios

Un beneficiario es alguien a quien podrás enviar dinero. Cada usuario tiene su propia lista. Para que una transferencia funcione, el `accountNumber` del beneficiario debe corresponder a una cuenta real de otro usuario que exista en el sistema.

POST `/beneficiaries`

Crea un beneficiario para el usuario autenticado.

- **Auth requerida:** sí
- **Body:**

```
{
  "name": "Ana",
  "lastName": "García",
  "accountNumber": "1234567890",
  "alias": "Ana del trabajo"
}
```

Campo	Tipo	Obligatorio	Descripción
name	string	sí	Nombre del beneficiario
lastName	string	sí	Apellido
accountNumber	string	sí	Número de cuenta DESTINO (la cuenta a la que enviarás)
alias	string	sí	Apodo para identificarlo en tu app

Respuesta exitosa (`201 Created`):

```
{
  "id": "ben_xyz123",
  "ownerId": "uid-del-usuario",
  "name": "Ana",
  "lastName": "García",
  "accountNumber": "1234567890",
  "alias": "Ana del trabajo",
  "createdAt": { "_seconds": 1717325400, "_nanoseconds": 0 }
}
```

Errores posibles:

Status	error	Cuándo
400	invalid_body	Falta uno de los campos o no es texto válido
401	unauthorized	Token faltante o inválido

GET /beneficiaries

Lista TODOS los beneficiarios del usuario autenticado.

- Auth requerida: sí

Respuesta exitosa (200 OK):

Un array (posiblemente vacío) de beneficiarios.

```
[
  {
    "id": "ben_xyz123",
    "ownerId": "uid",
    "name": "Ana",
    "lastName": "García",
    "accountNumber": "1234567890",
    "alias": "Ana del trabajo"
  }
]
```

GET /beneficiaries/{id}

Devuelve un beneficiario específico por su id .

- Auth requerida: sí
- Path param: id — el id del beneficiario (campo id devuelto en POST)

Respuesta exitosa (200 OK):

```
{
  "id": "ben_xyz123",
  "ownerId": "uid",
  "name": "Ana",
  "lastName": "García",
  "accountNumber": "1234567890",
  "alias": "Ana del trabajo"
}
```

Errores posibles:

Status	error	Cuándo
401	unauthorized	Token faltante o inválido
404	not_found	El beneficiario no existe O pertenece a otro usuario

Por seguridad, la API responde 404 también cuando el id existe pero es de otro usuario, para no revelar la existencia de recursos ajenos.

PUT /beneficiaries/{id}

Actualiza todos los campos de un beneficiario.

- Auth requerida: sí
- Path param: id

- **Body:** los cuatro campos completos (igual que en `POST`).

Respuesta exitosa (`200 OK`):

El beneficiario actualizado.

Errores posibles: mismos que en `POST` + `404 not_found`.

`DELETE /beneficiaries/{id}`

Borra un beneficiario.

- **Auth requerida:** sí
- **Path param:** `id`

Respuesta exitosa (`200 OK`):

```
{ "id": "ben_xyz123", "deleted": true }
```

Errores posibles: `401`, `404`.

Endpoints de transacciones

`POST /transaction`

Transfiere dinero de tu cuenta a la cuenta de uno de tus beneficiarios. La operación es **atómica**: o se descuenta de ti y se le acredita al otro, o no pasa nada.

- **Auth requerida:** sí
- **Body:**

```
{
  "toBeneficiaryId": "ben_xyz123",
  "amount": 2500,
  "description": "Pago de la renta"
}
```

Campo	Tipo	Obligatorio	Descripción
<code>toBeneficiaryId</code>	string	sí	<code>id</code> del beneficiario destino
<code>amount</code>	entero positivo	sí	Monto a transferir, en centavos
<code>description</code>	string	no	Concepto / descripción del movimiento

Respuesta exitosa (`201 Created`):

```
{
  "id": "tx_abc456",
  "ownerId": "uid-del-emisor",
  "fromAccount": "4827193056",
  "toAccount": "1234567890",
  "toBeneficiaryId": "ben_xyz123",
  "participants": ["4827193056", "1234567890"],
  "amount": 2500,
  "description": "Pago de la renta",
  "status": "completed",
  "date": { "_seconds": 1717325400, "_nanoseconds": 0 }
}
```

Errores posibles:

Status	error	Cuándo

400 Status	invalid_body error	Falta toBeneficiaryId Cuando
400	invalid_amount	amount no es entero positivo
400	insufficient_funds	No tienes saldo suficiente
400	same_account	El beneficiario apunta a tu propia cuenta
401	unauthorized	Token faltante o inválido
404	no_account	No tienes cuenta de origen creada
404	beneficiary_not_found	El beneficiario no existe o no es tuyo
404	dest_account_not_found	El beneficiario apunta a un accountNumber que no existe

Ejemplo:

```
apiCall {
  api.createTransaction(
    TransactionRequest(
      toBeneficiaryId = "ben_xyz123",
      amount = 2500,
      description = "Pago de la renta"
    )
  )
}.onFailure { e ->
  if (e is ApiException) {
    when (e.errorCode) {
      "insufficient_funds"      -> show("Saldo insuficiente")
      "beneficiary_not_found"  -> show("Beneficiario inválido")
      "dest_account_not_found" -> show("La cuenta destino no existe")
      "same_account"          -> show("No puedes enviarte a ti mismo")
    }
  }
}
```

GET /transaction

Lista los movimientos de tu cuenta, tanto los que **enviaste** como los que **recibiste**, ordenados del más reciente al más antiguo.

- **Auth requerida:** sí

Respuesta exitosa (200 OK):

```
[
  {
    "id": "tx_abc456",
    "ownerId": "uid-del-emisor",
    "fromAccount": "4827193056",
    "toAccount": "1234567890",
    "amount": 2500,
    "description": "Pago de la renta",
    "status": "completed",
    "date": { "_seconds": 1717325400, "_nanoseconds": 0 },
    "direction": "out"
  }
]
```

El campo extra `direction` te dice, desde **tu** punto de vista, si la transacción fue una salida o una entrada:

direction	Significado
"out"	Tú enviaste el dinero (mostrar en rojo, signo -)

direction	Significado
in	Tu recibiste el dinero (mostrar en verde, signo +)

Ojo: una transacción que tú envías y otro recibe aparece en el `GET` de ambos usuarios, con `direction` distinto según quién consulte.

Errores posibles:

Status	error	Cuándo
401	unauthorized	Token faltante o inválido
404	no_account	No tienes cuenta creada

Tabla de códigos de error

Referencia rápida de todos los códigos que puede devolver la API:

errorCode	Status	Significado
unauthorized	401	Token faltante, inválido o expirado
expired	410	La API educativa ya pasó su fecha de expiración
invalid_body	400	Falta algún campo obligatorio del body
invalid_amount	400	El monto no es un entero positivo
insufficient_funds	400	Saldo insuficiente para la transferencia
same_account	400	Intento de transferir a tu propia cuenta
no_account	404	El usuario no tiene cuenta creada
not_found	404	Beneficiario no encontrado
beneficiary_not_found	404	Beneficiario no encontrado (en contexto de transferencia)
dest_account_not_found	404	La cuenta destino del beneficiario no existe
account_exists	409	El usuario ya tiene una cuenta
internal_error	500	Error inesperado del servidor

Ejemplo de flujo completo

Escenario: un usuario quiere mandarle \$25.00 a su amiga Ana. El siguiente método de un `ViewModel1` encadena los pasos típicos.

```

class TransferViewModel(
    private val repo: BankingRepository // tu repositorio
) : ViewModel() {

    fun pagarAAna() {
        viewModelScope.launch {
            // 1) Asegurarse de tener cuenta. Si no, crearla.
            var account = apiCall { repo.getAccount() }.getOrElse { e ->
                if (e is ApiException && e.errorCode == "no_account") {
                    repo.createAccount()
                } else {
                    return@launch reportar(e)
                }
            }

            // 2) Si no hay saldo suficiente, fondear.
            if (account.balance < 2500) {
                account = apiCall { repo.fundAccount(10000) }
                    .getOrElse { return@launch reportar(it) }
            }

            // 3) Asegurarse de que Ana está en mis beneficiarios.
            val lista = apiCall { repo.listBeneficiaries() }
                .getOrElse { return@launch reportar(it) }

            val ana = lista.firstOrNull { it.accountNumber == "1234567890" }
                ?.apiCall {
                    repo.createBeneficiary(
                        name = "Ana", lastName = "García",
                        accountNumber = "1234567890", alias = "Ana"
                    )
                }.getOrElse { return@launch reportar(it) }

            // 4) Hacer la transferencia.
            apiCall {
                repo.createTransaction(
                    toBeneficiaryId = ana.id,
                    amount = 2500,
                    description = "Para el café"
                )
            }
                .onSuccess { tx -> Log.d("Banca", "Transferencia OK: ${tx.id}") }
                .onFailure { ::reportar }
        }
    }

    private fun reportar(e: Throwable) {
        // Mapear a tu UI (LiveData / StateFlow)
        Log.e("Banca", "Error: ${e.message}")
    }
}

```